



Page Replacement Algorithm Simulator



Shahariar Nadeem

CSE 4300



What is a Page Replacement Algorithm Simulator?

- Takes a list of page numbers and a number of frames
- Simulates how the operating system loads pages into memory
- Decides which page to remove when memory is full
- Counts how many **page faults** and **hits** happen
- Shows how the frames change at each step
- In this project, I wrote it in C and tested with following algorithms
 - FIFO
 - Random
 - LRU
 - MIN (Optimal)

Algorithms Implemented

- FIFO, First In, First Out, removes the page that has been in memory the longest
- Random removes a page chosen at random
- LRU, Least Recently Used, removes the page that hasn't been used for the longest time
- MIN, Optimal, removes the page that will be used farthest in the future

FIFO Algorithm

```
SimResult simulate_fifo(const int *refs, int n, int frames) {  
    SimResult res;  
    int *frame;  
    int next;  
    int i;  
  
    res.hits = 0;  
    res.faults = 0;  
  
    frame = (int *)malloc(frames * sizeof(int));  
    if (frame == NULL) {  
        return res;  
    }  
  
    init_frames(frame, frames);  
    next = 0;  
  
    for (i = 0; i < n; i++) {  
        int page = refs[i];  
        int pos = find_page(frame, frames, page);  
  
        if (pos != -1) {  
            res.hits++;  
        } else {  
            res.faults++;  
            frame[next] = page;  
            next = (next + 1) % frames;  
        }  
    }  
  
    free(frame);  
    return res;  
}
```

Random Algorithm

```
SimResult simulate_random(const int *refs, int n, int frames) {
    SimResult res;
    int *frame;
    int i;

    res.hits = 0;
    res.faults = 0;

    frame = (int *)malloc(frames * sizeof(int));
    if (frame == NULL) {
        return res;
    }

    init_frames(frame, frames);

    for (i = 0; i < n; i++) {
        int page = refs[i];
        int pos = find_page(frame, frames, page);

        if (pos != -1) {
            res.hits++;
        } else {
            int free_index = -1;
            int j;

            res.faults++;

            for (j = 0; j < frames; j++) {
                if (frame[j] == -1) {
                    free_index = j;
                    break;
                }
            }
        }
    }
}
```

LRU Algorithm

```
SimResult simulate_lru(const int *refs, int n, int frames) {
    SimResult res;
    int *frame;
    int *last_used;
    int i, t;

    res.hits = 0;
    res.faults = 0;

    frame = (int *)malloc(frames * sizeof(int));
    last_used = (int *)malloc(frames * sizeof(int));
    if (frame == NULL || last_used == NULL) {
        free(frame);
        free(last_used);
        return res;
    }

    init_frames(frame, frames);
    for (i = 0; i < frames; i++) {
        last_used[i] = -1;
    }

    for (t = 0; t < n; t++) {
        int page = refs[t];
        int pos = find_page(frame, frames, page);

        if (pos != -1) {
            res.hits++;
            last_used[pos] = t;
        } else {
            int free_index = -1;

            res.faults++;

            for (i = 0; i < frames; i++) {
                if (frame[i] == -1) {
```

Min (Optimal) Algorithm

```
SimResult simulate_min(const int *refs, int n, int frames) {
    SimResult res;
    int *frame;
    int t;

    res.hits = 0;
    res.faults = 0;

    frame = (int *)malloc(frames * sizeof(int));
    if (frame == NULL) {
        return res;
    }

    init_frames(frame, frames);

    for (t = 0; t < n; t++) {
        int page = refs[t];
        int pos = find_page(frame, frames, page);

        if (pos != -1) {
            res.hits++;
        } else {
            int free_index = -1;
            int i;

            res.faults++;

            for (i = 0; i < frames; i++) {
                if (frame[i] == -1) {
                    free_index = i;
                    break;
                }
            }
        }
    }
}
```

Logic

- For each page reference `refs[t]`, use `find_page(frame, frames, page)`, If the page is found then it hit
- If the page is not found then page fault, If there is an empty frame, put the page there. Otherwise, use the algorithm's rule, FIFO, Random, LRU, or MIN, to choose a frame to replace
- Each returns with total faults and hits

Terminal Outputs for Next Slides

Uses the following

Frames = 3

Page references = 8

Page numbers = 1 1 2 1 3 4 1 2



FIFO Algorithm Output

```
shah@DESKTOP-DGEE3NP:/mnt/c/Users/Shaha/Downloads/Project$ ./page_simulator
```

Algorithm: FIFO

Frames: 3

Total references: 8

Page faults: 6

Page hits: 2

Hit ratio: 0.2500

Fault ratio: 0.7500

FIFO trace:

Time	Ref	F0	F1	F2	H/F
------	-----	----	----	----	-----

-----+-----+-----+-----

0	1	1	-	-	F
---	---	---	---	---	---

1	1	1	-	-	H
---	---	---	---	---	---

2	2	1	2	-	F
---	---	---	---	---	---

3	1	1	2	-	H
---	---	---	---	---	---

4	3	1	2	3	F
---	---	---	---	---	---

5	4	4	2	3	F
---	---	---	---	---	---

6	1	4	1	3	F
---	---	---	---	---	---

7	2	4	1	2	F
---	---	---	---	---	---

Random Algorithm Output

```
shah@DESKTOP-DGEE3NP:/mnt/c/Users/Shaha/Downloads/Project$ ./page_simulator
```

Algorithm: LRU

Frames: 3

Total references: 8

Page faults: 5

Page hits: 3

Hit ratio: 0.3750

Fault ratio: 0.6250

LRU trace:

Time	Ref	F0	F1	F2	H/F
------	-----	----	----	----	-----

0	1	1	-	-	F
1	1	1	-	-	H
2	2	1	2	-	F
3	1	1	2	-	H
4	3	1	2	3	F
5	4	1	4	3	F
6	1	1	4	3	H
7	2	1	4	2	F

LRU Algorithm Output

```
shah@DESKTOP-DGEE3NP: /mnt/c/Users/Shaha/Downloads/Project$ ./page_simulator
```

Algorithm: LRU

Frames: 3

Total references: 8

Page faults: 5

Page hits: 3

Hit ratio: 0.3750

Fault ratio: 0.6250

LRU trace:

Time	Ref	F0	F1	F2	H/F
------	-----	----	----	----	-----

0	1	1	-	-	F
---	---	---	---	---	---

1	1	1	-	-	H
---	---	---	---	---	---

2	2	1	2	-	F
---	---	---	---	---	---

3	1	1	2	-	H
---	---	---	---	---	---

4	3	1	2	3	F
---	---	---	---	---	---

5	4	1	4	3	F
---	---	---	---	---	---

6	1	1	4	3	H
---	---	---	---	---	---

7	2	1	4	2	F
---	---	---	---	---	---

Algorithm: MTN (Optimal)

Min (optimal) Algorithm Output

```
shah@DESKTOP-DGEE3NP:/mnt/c/Users/Shaha/Downloads/Project$ ./page_simulator
```

Algorithm: MIN (Optimal)

Frames: 3

Total references: 8

Page faults: 4

Page hits: 4

Hit ratio: 0.5000

Fault ratio: 0.5000

MIN trace:

Time	Ref	F0	F1	F2	H/F
------	-----	----	----	----	-----

-----+-----+-----+-----

0	1	1	-	-	F
---	---	---	---	---	---

1	1	1	-	-	H
---	---	---	---	---	---

2	2	1	2	-	F
---	---	---	---	---	---

3	1	1	2	-	H
---	---	---	---	---	---

4	3	1	2	3	F
---	---	---	---	---	---

5	4	1	2	4	F
---	---	---	---	---	---

6	1	1	2	4	H
---	---	---	---	---	---

7	2	1	2	4	H
---	---	---	---	---	---

Benefits of this Simulator

- Can see pages moving in and out of frames
- Be able to see it as trace tables
- Easy to compare algorithms. Same frames and same reference string for all four. You can directly see which one has more hits and faults
- You can change the number of frames or references and run again and see the results with different inputs

Thank you!

